

Backend & Database Master Plan

Assignee: Adham (Lead Backend Developer)

Project: Hotel Booking System

UI Framework: Java Swing (Managed by Alaa & Hanin)

Deadline: Phase 1 must be completed within 4-5 days.

Your Objective

As the backend developer, you are building the "engine" of the hotel system. You do **NOT** need to worry about the GUI (JFrame, JPanel, etc.). Your sole responsibility is to translate Sherif's UML class diagram into pure, error-free Java code and connect it to a relational database using JDBC.

The UI team (Alaa & Hanin) will rely entirely on the methods you write to make the buttons in their Swing screens actually work.

Phase 1: Database Setup (Schema)

We are moving away from basic text files to a real database. **SQLite** is highly recommended for this project as it doesn't require setting up a local server like MySQL, making it perfect for desktop Swing apps.

You need to create three main tables in your database:

1. **users table:** user_id (PK), name, email, password, role (admin/customer), and phone_number (null for admins).
2. **rooms table:** room_id (PK), room_number, type (single/double/suite), price, is_available (Boolean/Integer 0 or 1).
3. **bookings table:** booking_id (PK), customer_id (FK), room_id (FK), check-in date, check-out date, amount, and status.

Phase 2: Core OOP Models (The Entities)

You must translate Sherif's UML diagram exactly into Java classes.

- Create the abstract class User and let Admin and Customer inherit from it.
- Create the Room class and the Booking class.
- Create the interface IReservable and make sure the Room class implements it.
- **Rule:** Keep all attributes private and generate getters and setters (encapsulation).

Phase 3: The Database Connection (Singleton Pattern)

To prevent the application from opening too many connections to the database (which causes memory leaks), you must use the **singleton design pattern**.

Task: Create a class named DatabaseConnection.

- Make its constructor private.
- Create a private static DatabaseConnection instance.
- Create a public static Connection getConnection() method.

Phase 4: The DAO Pattern (Crucial Step!)

DAO (Data Access Object) is an architectural pattern. Do NOT write SQL queries inside your Room or User classes. Instead, create separate DAO classes.

Here is the exact **"Method Contract"** you must write. The UI team will be calling these exact methods, so keep the names exactly as listed below:

1. UserDAO Class

Handles all login and registration logic.

- public User loginUser(String email, String password)
(Returns a Customer or Admin object if successful, or null if failed)
- public boolean registerCustomer(Customer customer)

2. RoomDAO Class

Handles hotel room data.

- public List<Room> getAllRooms()
(UI team needs this to populate the JTable for the admin.)
- public List<Room> getAvailableRooms()
(UI team needs this for the Customer search screen.)
- public boolean addRoom(Room room)
- public boolean updateRoomStatus(int roomId, boolean isAvailable)

3. BookingDAO Class

Handles the reservation process.

- public boolean createBooking(Booking booking)
(Note: This method must also call updateRoomStatus to mark the room as unavailable!)
- public List<Booking> getBookingsByCustomer(int customerId)
- public List<Booking> getAllBookings()

Adham's Golden Technical Rules:

1. **Use PreparedStatement ONLY:** Never use normal statements for queries. A prepared statement protects the app from SQL injection attacks and handles variables more cleanly.
2. **Close Your Resources:** Always close your ResultSet, PreparedStatement, and Connection in a finally block or use try-with-resources to prevent memory leaks.
3. **Custom Exceptions:** If a database error occurs, don't just print it to the console. Throw a custom exception (e.g., DatabaseException) so the UI team can catch it and show a JOptionPane error message to the user.

Next Steps for You:

Set up your IDE, add the JDBC SQLite/MySQL .jar file to your project libraries, and start coding the models and DAO. Ping Youssef or the UI team once your DAO methods are ready to be tested! You got this!